Data Pull

Tags	storage object inspector, storage run_cu_query.py, unsupported pic event_scheduler, not processing c message, JSON, commands out c STORING DATA
Team Owner	WHAT
Last Audit	ACTIVE
Last Modified	Jan 6, 2021

- Introduction
- Diagnosis and Mitigation
  - JSON data pull
  - turning on storage logging
  - storage object inspection with do_gather_storage_objects.sh
    - Intro
    - run_cu_query.py: running queries on one of more CUs from the CDB
    - command line: overview
    - command line: basics
    - command line: ENV_OPT_STORE
    - command line: ENV_OPT_PYTHON, ENV_OPT_STORE, and storage object protocols pickle versus json versus json2
    - command line: ENV_OPT_QUEUE, low frequency queue
    - command line: ENV_OPT_QUEUE, high frequency queue
    - command line: ENV_OPT_TYPE
    - filter on task key

    - storing an object and reviewing queries and debug output
    - storage object types and attributes of interest
  - storage object inspection using run_cu_query.py and /opt/em7/backend/storage_obj_inspector.py
    - Intro
    - finding storage object ids from the meta tables
    - inspecting storage objects with run_cu_query.py
    - enabling the storage object inspector
    - running the storage object inspector
    - using run_cu_query.py to retrieve and run storage objects: retrieving additional attributes
    - Recipe: Pull and run the most recent PerfDynamicAppStorage
    - Recipe: gather a list of N storage objects and their attributes
    - Recipe: gather the N most recent storage objects with the given task key pattern
    - Recipe: run one of the storage objects you just gathered
    - Postmortem Examples
  - worker processes with high CPU utilisation
  - look for timeouts
  - slow storage object logging
- Theory
  - in_storage.X_storage tables
- Metrics
- Post Mortem Articles
  - 2025-03-12 EM-72817: Lots of "Storage Object Failure due to ERROR STORING DATA." after an upgrade from 12.1.2 to 12.3.1
  - 2024-10-11 EM-70585: Investigate "not processing deprecated messages events"
  - 2024-06-11 EM-65237: aws-kd-mc-eads-elv-2-sl1-de-01-prod - dl09pavsl1009 medium frequency is 128550 behind (threshold: 120000)
  - 2024-05-07 EM-64857: Escalation for case 00435646
  - 2024-04-04 EM-64173: v12.1.1 SL1 upgrade issues - Singledigits03-prod
  - 2024-03-29 EM-64099: Degraded performance in HF data pull due to task_key "0" in storage objects
  - 2022-11-10 EM-54550 Bandwidth Billing Isn't Working
- Known Issues
- Silo Log Messages
  - Introduction
  - analysis script
  - Log Pattern Analysis
    - stanza:timeout

Introduction

Diagnosis and Mitigation

JSON data pull

See this page: JSON Datapull - Dev Debugging Guide

turning on storage logging

To see the silo.logger.debug() messages from storage object code, you need to add this section to /etc/silo.conf

```
[DATAPULL]
```

```
log_storage = 1

```

and then restart the hf, lf, or mf data pull process you want to instrument. For example:

```
[em7admin@cdb ~]$ sudo bash
[sudo] password for em7admin:
[root@cdb em7admin]# systemctl | grep pull
  em7_hfpulld.service                                                          loaded a
  em7_lfpulld.service                                                          loaded a
  em7_mfpulld.service                                                          loaded a

[root@cdb em7admin]# systemctl restart em7_mfpulld.service
```

This will generate a ton of log messages in /data/logs/silo.log, so you typically only want to do it for short periods of time. If all you need is to see a particular type of storage object run once, see the section on the storage object inspector below.

storage object inspection with do_gather_storage_objects.sh

Intro

The following two scripts:

https://github.sciencelogic.com/SL1/em7_diag/blob/main/run_cu_query.py

https://github.sciencelogic.com/SL1/em7_diag/blob/main/do_gather_storage_objects.sh

run_cu_query.py

do_gather_storage_objects.sh

can be used on a CDB to:

- retrieve a storage objects from collectors
- print information about them (dump their data vectors and other contents)
- execute storage objects with debug level messages written to stdout

run_cu_query.py: running queries on one of more CUs from the CDB

The python program run_cu_query.py can be used from the CDB to run a query against a single CU or against all of them, and retrieve the results. Invoked without arguments, we see this syntax guide:

```
./run_cu_query.py
usage: command [options]
   valid commands are:
        get_obj2
        get_obj
        run
        run_sep_files
        log_info
```

```
        mids
        fetch_last_ifpoll

```

The run command run with the --help argument, offers this syntax guide:

```
./run_cu_query.py run --help
usage: run_cu_query.py [-h] [--mid MID] [--delimiter DELIMITER] query

positional arguments:
  query

optional arguments:
  -h, --help            show this help message and exit
  --mid MID
  --delimiter DELIMITER

       run a query against all CUs and MCUs in the system
          a common use of this is to gather the recent N storage objects from all collectors,
             ./run_cu_query.py run "select seq, collection_time, type, task_key, policy_info

          or a simpler version, that only goes against one CU (in this case with module id 5)
             ./run_cu_query.py run "select * from in_storage.mf_storage_meta order by collect

```

When run without a --mid argument, the query text is run against all appliances and the results are returned, comma seperated, with the module id as the first column:

```
./run_cu_query.py run "select id,function,name,ip from master.system_settings_licenses"
3,3,5,cu1,10.2.4.52
4,4,5,cu2,10.2.4.53
5,5,6,mcu2,10.2.4.54
```

In this case, each CU (the mdule ids are 3, 4, and 5) had one row in its local master.systems_settings_licenses

command line: overview

run on a CDB or DE, do_gather_storage_objects.sh uses the run_cu_query.py program to retrieve storage object data from these tables on CUs

- in_storage.{lf,mf,hf}_storage_meta
- in_storage.{lf,mf,hf}_storage

and gather it into a tarball under /tmp. By default, it retrieves data from the mf queue tables. The query against the storage_object_meta table is written to a single file, but the queries against the storage table are written to separate files (one per storage object) and include more information (pickle of the python storage object is retrieved, unpickled, and data from the object attributes is printed).

the syntax guide is (printed when invoked without arguments):

```
usage: <mid> <where> <limit> <password>
```

command line: basics

The commands shown here currently (2025-03-20) work for EM7 versions after 12.x (I don't know the exact version) and before 12.3.1. There are options you have to set to handle versions before 12.x and at 12.1.3 or after. You can read about these in this section: Data Pull | command line: ENV_OPT_PYTHON, ENV_OPT_STORE, and storage object protocols pickl... .

Here is the simplest useful command line:

```
./do_gather_storage_objects.sh 0 '%' 3 em7admin
```

so we have set:

- mid=0, which means to run the queries against all CUs
- where='%', which forms the where clause for the query against the storage_meta table as: where task_key like '%'
- limit=3, which means to retrieve the most recent 3 storage objects
- password=em7admin (this is the password to the em7admin account on the CDB, which is used in the bash script to run a sudo command)

Here is the stdout from running this command line on my test stack (10.2.4.50):

```
[em7admin@cdb gather]$ ./do_gather_storage_objects.sh 0 '%' 3 em7admin
retrieving seq_id 1279649 from mid 3
retrieving seq_id 1279648 from mid 3
retrieving seq_id 1279647 from mid 3
retrieving seq_id 232 from mid 4
retrieving seq_id 231 from mid 4
retrieving seq_id 230 from mid 4
retrieving seq_id 233 from mid 5
retrieving seq_id 232 from mid 5
retrieving seq_id 231 from mid 5
creating tarball
tar: Removing leading / from member names
/tmp/gather_storage_20240429_200556/
/tmp/gather_storage_20240429_200556/run_cu_query.py
/tmp/gather_storage_20240429_200556/report_meta.txt
/tmp/gather_storage_20240429_200556/run_mid_3_seq_1279649.txt
/tmp/gather_storage_20240429_200556/run_mid_3_seq_1279648.txt
/tmp/gather_storage_20240429_200556/run_mid_3_seq_1279647.txt
/tmp/gather_storage_20240429_200556/run_mid_4_seq_232.txt
/tmp/gather_storage_20240429_200556/run_mid_4_seq_231.txt
/tmp/gather_storage_20240429_200556/run_mid_4_seq_230.txt
/tmp/gather_storage_20240429_200556/run_mid_5_seq_233.txt
/tmp/gather_storage_20240429_200556/run_mid_5_seq_232.txt
/tmp/gather_storage_20240429_200556/run_mid_5_seq_231.txt
all done, please retrieve, and then delete: /tmp/gather_storage_20240429_200556.tgz
```

In the tarball the file report_meta.txt contains the results of the query against the meta table on each CU, while each of these files:

```
run_mid_{}_seq_{}.txt
```

contains information retrieved from the in_storage.{lf,mf,hf}_storage table for one storage object. Continuing the example above, report_meta.txt has these contents:

```
3,1279649,2024-04-29 20:05:32,alert_count_storage,alert_count_storage,alert_count_storage
3,1279648,2024-04-29 20:05:24,PerfDynamicAppStorage,DynamicApp:1:1001,(APP: 1001 DID: 1)
3,1279647,2024-04-29 20:05:24,PerfDynamicAppStorage,DynamicApp:1:1002,(APP: 1002 DID: 1)
4,232,2024-04-29 00:53:14,alert_count_storage,alert_count_storage,alert_count_storage
4,231,2024-04-29 00:52:09,system_message_storage,('system_message_storage', (4,)),system_messa
4,230,2024-04-29 00:52:08,system_message_storage,('system_message_storage', (4,)),system_messa
5,233,2024-04-29 00:52:59,alert_count_storage,alert_count_storage,alert_count_storage
5,232,2024-04-29 00:52:01,system_message_storage,('system_message_storage', (5,)),system_messa
5,231,2024-04-29 00:52:01,system_message_storage,('system_message_storage', (5,)),system_messa
```

The line format here is:

```
<mid>,<seq>,<collection_time>,<type>,<task_key>,<policy_info>
```

The information gathered from queries against the in_storage.{lf,mf,hf}_storage tables are written into separate files with this naming convention

```
run_mid_{}_seq_{}.txt
```

for example, in run_mid_4_seq_230.txt we have:

```
did=0, t=2024-04-29 00:52:08, data=('silo.maint_daily.4: pruner knob pruner_minRowLimit value
"2024-04-29 20:06:03,670","DEBUG","19176","silo.storage_obj_inspector.255","proc_pda","560","t
"2024-04-29 20:06:03,671","DEBUG","19176","silo.storage_obj_inspector.255","proc_pda","577","S
"2024-04-29 20:06:03,671","DEBUG","19176","silo.storage_obj_inspector.255","proc_pda","339","L
```

The first line prints the following data from the unpickled storage object x

- x.did (0 in this case, meaning the storage object was not aligned with a device)
- x.collection time
- x.data (the message to be stored in this case)

command line: ENV_OPT_STORE

One problem with our first command line is that some storage objects (like PerfDynamicAppStorage) don't use the default x.data attribute at all and store their data in other attributes, that vary by storage object type.

Continuing with our example, we use a more complicated command line:

```
ENV_OPT_STORE="--obj-attributes=mid,alert_counts,data_by_obj_by_idx,derived_rates" ./do_gather
```

ENV_OPT_STORE is being used to specify additional object attributes to fetch. do_gather_storage_objects.sh passes them on to run_cu_query.py.

Running that command line, produces gather_storage_20240430_131510.tgz, with this report_meta.txt:

```
3,1293310,2024-04-30 13:14:12,PerfDynamicAppStorage,DynamicApp:1:859,(APP: 859 DID: 1)
3,1293309,2024-04-30 13:14:12,PerfDynamicAppStorage,DynamicApp:2:868,(APP: 868 DID: 2)
3,1293308,2024-04-30 13:14:12,PerfDynamicAppStorage,DynamicApp:1:868,(APP: 868 DID: 1)
4,235,2024-04-30 00:54:34,alert_count_storage,alert_count_storage,alert_count_storage
4,234,2024-04-30 00:52:10,system_message_storage,('system_message_storage', (4,)),system_mess
4,233,2024-04-30 00:52:09,system_message_storage,('system_message_storage', (4,)),system_mess
5,236,2024-04-30 00:54:25,alert_count_storage,alert_count_storage,alert_count_storage
5,235,2024-04-30 00:52:01,system_message_storage,('system_message_storage', (5,)),system_mess
5,234,2024-04-30 00:52:01,system_message_storage,('system_message_storage', (5,)),system_mess

```

and this run_mid_3_seq_1293308.txt file showing the value of the attributes data_by_obj_by_idx and derived_rates:

```
did=1, t=2024-04-30 13:14:12, data=None
data_by_obj_by_idx={8704: {'.1': (u'Scan', 0, 7.370948791503906, '.1'), '.2': (u'Steal', 0, 7.
derived_rates=defaultdict(<type 'dict'>, {'.1': {8705: 0.0, 8707: 0.0, 8721: 0.0}, '.2': {8705
...
```

and this run_mid_4_seq_235.txt, showing the values of mid and alert_counts:

```
ignore="
did=0, t=2024-04-30 00:54:34, data=None
mid=4
alert_counts=defaultdict(<type 'dict'>, {(0, 0, None, 0, 2): {2: 2}})
...
```

We have written a section below to gather the attributes of different storage object types for reference.

command line: ENV_OPT_PYTHON, ENV_OPT_STORE, and storage object protocols pickle versus json versus json2

Storage objects used to be serialized (converted into a sequence of bytes that could be written to a database or file) via python's pickle protocol. At some point in the 12.x line, this was changed to a json format. Then, in version 12.3.1, the location of some of the various code segments used to deserialize storage objects changed yet again. Also in version 12.3.1, some of the code because python3 only.

To accomodate this, run_cu_query.py's get_obj and get_obj2 commands were then modified to handle these variations via these three command line options

- --so-prot=pickle (for versions before 12.x, running under python2)
- --so-prot=json (for versions > =12.x and <12.3.1, running under python2)

- --so-prot=json2 (for versions >= 12.3.1, running under python3)

The default is currently set to “json”. So if you are using run_cu_query.py and do_gather_storage_objects.sh on a stack running an older version of sl1 that uses the pickle protocol, you will need to specify the option

```
--so-prot=pickle
```

If you are using the do_gather_storage_objects.sh script, you can achieve this by adding that option to the ENV_OPT_STORE variable like this:

```
ENV_OPT_STORE="--obj-attributes=mid,alert_counts,data_by_obj_by_idx,derived_rates --so-prot=pi
```

If you are using the do_gather_storage_objects.sh script on a 12.3.1 or newer system, you will want to use this command line:

```
ENV_OPT_PYTHON="python3" ENV_OPT_STORE="--obj-attributes=mid,alert_counts,data_by_obj_by_idx,d
```

If the 12.3.1 system was recently upgraded, there may be CU’s where the latest storage objects use an older protocol. Those will fail to deserialize. Here is some of the output from running the above command line on such a system:

```
retrieving seq_id 878358 from mid 10
[sudo] password for em7admin: /opt/em7/lib/python3/OpenSSL/crypto.py:8: CryptographyDeprecati
  from cryptography import utils, x509
retrieving seq_id 878351 from mid 10
[sudo] password for em7admin: /opt/em7/lib/python3/OpenSSL/crypto.py:8: CryptographyDeprecati
  from cryptography import utils, x509
retrieving seq_id 878348 from mid 10
[sudo] password for em7admin: /opt/em7/lib/python3/OpenSSL/crypto.py:8: CryptographyDeprecati
  from cryptography import utils, x509
retrieving seq_id 2188073 from mid 11
[sudo] password for em7admin: /opt/em7/lib/python3/OpenSSL/crypto.py:8: CryptographyDeprecati
  from cryptography import utils, x509
retrieving seq_id 2183289 from mid 11
[sudo] password for em7admin: /opt/em7/lib/python3/OpenSSL/crypto.py:8: CryptographyDeprecati
  from cryptography import utils, x509
Unhandled exception: Traceback (most recent call last):
  File "/opt/em7/lib/python3.6/proc_pda.py", line 222, in __init__
  File "./run_cu_query.py", line 204, in main
    obj= decode_object(data, args.so_prot, self.logger)
  File "./run_cu_query.py", line 65, in decode_object
    return STORAGE_OBJECT_DISPATCH_MAP[dsm.storage_class_name][1][UNPACK_HANDLER](logger, dsm
  File "/opt/em7/lib/python3.6/datapull_storage.py", line 206, in unpack
  File "/opt/em7/lib/python3.6/unpack_defaults.py", line 18, in unpack_default
  File "/usr/lib64/python3.6/json/__init__.py", line 367, in loads
    return cls(**kw).decode(s)
  File "/usr/lib64/python3.6/json/decoder.py", line 339, in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
  File "/usr/lib64/python3.6/json/decoder.py", line 355, in raw_decode
    obj, end = self.scan_once(s, idx)
  File "/opt/em7/lib/python3.6/multi_decoder.py", line 39, in object_hook
  File "/opt/em7/lib/python3.6/dictkey_decoder.py", line 91, in object_hook
```

```
  File "/opt/em7/lib/python3.6/dictkey_decoder.py", line 91, in <dictcomp>
  File "/opt/em7/lib/python3.6/dictkey_decoder.py", line 65, in scan_key
TypeError: Invalid Key: msg
...
```

We see that three fetches against module id's 10 succeeded and two against module 11 worked, but the third failed. Looking at this files gathered in the tarball:

```
ls -la
total 56
drwxrwxr-x 1 chris chris   486 Mar 20 11:54 .
drwxrwxr-x 1 chris chris    60 Mar 20 12:02 ..
-rw-r----- 1 chris chris   975 Mar 20 11:54 report_meta.txt
-rwxr-xr-x 1 chris chris 15984 Mar 20 11:54 run_cu_query.py
-rw-r----- 1 chris chris   709 Mar 20 11:54 run_mid_10_seq_878348.txt
-rw-r----- 1 chris chris   709 Mar 20 11:54 run_mid_10_seq_878351.txt
-rw-r----- 1 chris chris   688 Mar 20 11:54 run_mid_10_seq_878358.txt
-rw-r----- 1 chris chris  3012 Mar 20 11:54 run_mid_11_seq_2183288.txt
-rw-r----- 1 chris chris  3012 Mar 20 11:54 run_mid_11_seq_2183289.txt
-rw-r----- 1 chris chris   709 Mar 20 11:54 run_mid_11_seq_2188073.txt
-rw-r----- 1 chris chris  3012 Mar 20 11:54 run_mid_9_seq_30.txt
-rw-r----- 1 chris chris  3012 Mar 20 11:54 run_mid_9_seq_31.txt
-rw-r----- 1 chris chris  3012 Mar 20 11:54 run_mid_9_seq_32.txt

```

we look at report_meta.txt to see when each of the storage objects was created:

```
cat report_meta.txt
10,878358,2025-03-20 03:20:16,system_message_storage,('system_message_storage', (10,)),system
10,878351,2025-03-20 03:20:12,system_message_storage,('system_message_storage', (10,)),system
10,878348,2025-03-20 03:20:12,system_message_storage,('system_message_storage', (10,)),system
11,2188073,2025-03-20 03:20:09,system_message_storage,('system_message_storage', (11,)),syste
11,2183289,2025-03-20 02:09:39,system_message_storage,('system_message_storage', (11,)),syste
11,2183288,2025-03-20 02:09:39,system_message_storage,('system_message_storage', (11,)),syste
9,32,2025-02-21 01:11:09,system_message_storage,('system_message_storage', (9,)),system_messa
9,31,2025-02-21 01:10:59,system_message_storage,('system_message_storage', (9,)),system_messa
9,30,2025-02-21 01:06:12,system_message_storage,('system_message_storage', (9,)),system_messa

```

TBD

command line: ENV_OPT_QUEUE, low frequency queue

In this variation, we look at storage objects in the low frequency data pull queue. Running this command line:

```
ENV_OPT_QUEUE="lf" ./do_gather_storage_objects.sh 0 '%' 3 em7admin
```

produces gather_storage_20240430_133911.tgz, with this report_meta.txt

```
3,3736393,2024-04-30 13:39:10,pda_tracker_storage,('pda_tracker_storage', (11, 14357)),pda_tra
3,3736392,2024-04-30 13:39:10,pda_tracker_storage,('pda_tracker_storage', (11, 14357)),pda_tra
3,3736391,2024-04-30 13:39:09,pda_tracker_storage,('pda_tracker_storage', (11, 14520)),pda_tra
4,1519813,2024-04-30 13:39:06,pda_tracker_storage,('pda_tracker_storage', (12, 21826)),pda_tra
4,1519812,2024-04-30 13:39:06,pda_tracker_storage,('pda_tracker_storage', (12, 21826)),pda_tra
4,1519811,2024-04-30 13:39:06,pda_tracker_storage,('pda_tracker_storage', (12, 21826)),pda_tra
```

```
5,2247,2024-04-30 13:00:03,pda_tracker_storage,('pda_tracker_storage', (1, 23986)),pda_tracker
5,2246,2024-04-30 12:00:03,pda_tracker_storage,('pda_tracker_storage', (1, 23986)),pda_tracker
5,2245,2024-04-30 11:00:00,pda_tracker_storage,('pda_tracker_storage', (1, 23986)),pda_tracker
```

and this run_mid_3_seq_3736393.txt:

```
did=0, t=2024-04-30 13:39:10, data={'pid': 14357, 'mid': 3, 'sp_id': 11, 'end_time': datetime.
...
```

command line: ENV_OPT_QUEUE, high frequency queue

Running this command line

```
ENV_OPT_QUEUE="hf" ./do_gather_storage_objects.sh 0 '%' 3 em7admin
```

produced this report_meta.txt:

```
3,89286,2024-04-30 13:43:13,event_dynamic,1,event_dynamic
3,89285,2024-04-30 13:42:13,event_dynamic,1,event_dynamic
3,89284,2024-04-30 13:41:13,event_dynamic,1,event_dynamic
4,11403,2024-04-30 13:30:57,system_status_output_storage,system_status_output_storage,system_s
4,11402,2024-04-30 13:30:23,needs_restart_storage,needs_restart_storage,needs_restart_storage
4,11401,2024-04-30 13:15:52,system_status_output_storage,system_status_output_storage,system_s
5,11402,2024-04-30 13:30:45,system_status_output_storage,system_status_output_storage,system_s
5,11401,2024-04-30 13:30:04,needs_restart_storage,needs_restart_storage,needs_restart_storage
5,11400,2024-04-30 13:15:44,system_status_output_storage,system_status_output_storage,system_s
```

However no interesting data was gathered for the event_dynamic objects:

```
cat run_mid_3_seq_89286.txt
did=1, t=2024-04-30 13:43:13, data=None
...
```

If we modify the command line to include the attributes used by event_dynamic objects:

```
 ENV_OPT_QUEUE="hf" ENV_OPT_STORE="--obj-attributes=matching_ders,event_id,event_defs,record"
```

we now see this report_meta.txt

```
3,89294,2024-04-30 13:49:13,event_dynamic,1,event_dynamic
3,89293,2024-04-30 13:48:13,event_dynamic,1,event_dynamic
3,89292,2024-04-30 13:47:13,event_dynamic,1,event_dynamic
4,11404,2024-04-30 13:45:53,system_status_output_storage,system_status_output_storage,system_s
4,11403,2024-04-30 13:30:57,system_status_output_storage,system_status_output_storage,system_s
4,11402,2024-04-30 13:30:23,needs_restart_storage,needs_restart_storage,needs_restart_storage
5,11403,2024-04-30 13:45:42,system_status_output_storage,system_status_output_storage,system_s
5,11402,2024-04-30 13:30:45,system_status_output_storage,system_status_output_storage,system_s
5,11401,2024-04-30 13:30:04,needs_restart_storage,needs_restart_storage,needs_restart_storage
```

and this in run_mid_3_seq_89294.txt:

```
did=1, t=2024-04-30 13:49:13, data=None
event_id=0
event_defs=None
record={'xtype': 1, 'seq': 70878, 'ip': '10.2.4.50', 'msg_id': 1007, 'auto_clear': 1, 'alert_i
...
```

command line: ENV_OPT_TYPE

If you were only interested in seeing event_dynamic objects, you could use this command line:

```
ENV_OPT_TYPE="event_dynamic" ENV_OPT_QUEUE="hf" ENV_OPT_STORE="--obj-attributes=matching_ders,
```

filter on task key

The task key for a PerfDynamicAppStorage object looks like this:

```
DynamicApp:1:859
```

if we only wanted to see PerfDynamicAppStorage objects for app 859, we could use this command

```
ENV_OPT_TYPE="PerfDynamicAppStorage" ENV_OPT_STORE="--obj-attributes=mid,alert_counts,data_by_
```

storing an object and reviewing queries and debug output

Adding the argument "--store" to the ENV_OPT_STORE variable will cause run_cu_query.py to execute the storage object after unpickling it, which produces very useful debug logs.

Running this command line:

```
ENV_OPT_TYPE="PerfDynamicAppStorage" ENV_OPT_STORE="--store --obj-attributes=mid,alert_counts,
```

produced this output, captured in the file run_mid_3_seq_1295842.txt (we have edited to only show some examples of the output available):

```
"2024-04-30 16:25:10,890","DEBUG","9243","silo.storage_obj_inspector.255","dynamic_app_storag
"2024-04-30 16:25:10,891","DEBUG","9243","silo.storage_obj_inspector.255","dynamic_app_storag
"2024-04-30 16:25:10,892","DEBUG","9243","silo.storage_obj_inspector.255","database","186","Q
                  app.name, app.app_type,
                  IF(afo.freq IS NOT NULL, afo.freq, app.poll) AS poll,
                  app.comp_dev, app.disabled_rollup, app.cu_affinity,
                  align.triggered_id AS triggered_id
                FROM master.map_dynamic_app_device_cred AS mdadc
                LEFT JOIN master.dynamic_app AS app
                  ON mdadc.app_id = app.aid
                LEFT JOIN
                  (SELECT DISTINCT triggered_id FROM master.dynamic_app_comp_autoalign) AS al
                  ON mdadc.app_id = align.triggered_id
                LEFT JOIN master.dynamic_app_freq_overrides AS afo
                  ON mdadc.app_id = afo.app_id AND mdadc.did = afo.did
                WHERE mdadc.app_id = 859 AND mdadc.did = 1"
"2024-04-30 16:25:10,893","DEBUG","9243","silo.storage_obj_inspector.255","database","187","Q
"2024-04-30 16:25:10,893","DEBUG","9243","silo.storage_obj_inspector.255","database","188","Q
"2024-04-30 16:25:10,893","DEBUG","9243","silo.storage_obj_inspector.255","database","189","Q
"2024-04-30 16:25:10,893","DEBUG","9243","silo.storage_obj_inspector.255","database","190","Q
"2024-04-30 16:25:10,893","DEBUG","9243","silo.storage_obj_inspector.255","database","186","Q
"2024-04-30 16:25:10,894","DEBUG","9243","silo.storage_obj_inspector.255","database","187","Q
"2024-04-30 16:25:10,894","DEBUG","9243","silo.storage_obj_inspector.255","database","188","Q
"2024-04-30 16:25:10,894","DEBUG","9243","silo.storage_obj_inspector.255","database","189","Q
"2024-04-30 16:25:10,894","DEBUG","9243","silo.storage_obj_inspector.255","database","190","Q
"2024-04-30 16:25:10,895","DEBUG","9243","silo.storage_obj_inspector.255","database","186","Q
```

```
              monitor_config, factor, string_type, trend_col, formula
            FROM master.dynamic_app_objects
            WHERE app_id=859"
"2024-04-30 16:25:10,895","DEBUG","9243","silo.storage_obj_inspector.255","database","187","Q
"2024-04-30 16:25:10,895","DEBUG","9243","silo.storage_obj_inspector.255","database","188","Q
"2024-04-30 16:25:10,895","DEBUG","9243","silo.storage_obj_inspector.255","database","189","Q
"2024-04-30 16:25:10,895","DEBUG","9243","silo.storage_obj_inspector.255","database","190","Q
"2024-04-30 16:25:10,896","DEBUG","9243","silo.storage_obj_inspector.255","common","213","App
"2024-04-30 16:25:10,896","DEBUG","9243","silo.storage_obj_inspector.255","perfconf_common","
"2024-04-30 16:25:10,896","DEBUG","9243","silo.storage_obj_inspector.255","perfconf_common","
"2024-04-30 16:25:10,897","DEBUG","9243","silo.storage_obj_inspector.255","database","186","Q
        FROM master.dynamic_app_index_map
        WHERE did=1 AND app_id=859"
"2024-04-30 16:25:10,897","DEBUG","9243","silo.storage_obj_inspector.255","database","187","Q
"2024-04-30 16:25:10,897","DEBUG","9243","silo.storage_obj_inspector.255","database","188","Q
"2024-04-30 16:25:10,897","DEBUG","9243","silo.storage_obj_inspector.255","database","189","Q
"2024-04-30 16:25:10,897","DEBUG","9243","silo.storage_obj_inspector.255","database","190","Q
...
```

storage object types and attributes of interest

```
type                          queue   attributes
PerfDynamicAppStorage         mf      data_by_obj_by_idx,derived_rates
alert_count_storage           mf      mid,alert_counts
event_dynamic                 hf      matching_ders,event_id,event_defs,record
pda_tracker_storage           lf      data
component_dev_storage         mf      data
dcm_avail_storage             mf      data
```

storage object inspection using run_cu_query.py and /opt/em7/backend/storage_obj_inspector.py

Intro

The sections below were written to document how to use

- the python script deployed on CDBs at /opt/em7/backend/storage_obj_inspector.py
- the python script run_cu_query.py , a more general script that overlaps with storage_obj_inspector.py

to retrieve information about storage objects from collectors. The command lines presented are clumsy and low level compared to the sections above, which describe how to use the script do_gather_storage_objects.sh to gather the same information.

When working with these command lines, you would typically need to retrieve info from in_storage.X_storage_meta first. The run_cu_query.py script is especially useful in this regard. We walk through an example of this here. You may want to review the schema for the in_storage tables in the theory section below before reading on.

Here are some other TSG sections where these scripts are discussed:

- Dynamic App Collection | debugging a config DA using storage object inspection

finding storage object ids from the meta tables

This utility:

```
https://code.eng.sciencelogic.com/projects/TOOL/repos/em7_diag/browse/run_cu_query.py

```

when dropped on a CDB can be used to run a query against all CUs at once, or against a single CU. We are going to use it to figure out the sequence id of an object to pull and run with storage_obj_inspector.py

When run without arguments:

```
[em7admin@cdb ~]$ ./run_cu_query.py
usage: command [options]
   valid commands are:
        run
        mids
        run_sep_files

```

It prints a usage guide. If you run the mids command, it will print a list of collector ids:

```
[em7admin@cdb ~]$ ./run_cu_query.py mids
5 6 7

```

If you have a did of X and you want to know the mid of the collector to target, you can just run this query:

```
select pid from master.legend_device where id=X
```

Here, we ask for the most recent 1000 rows of mf_storage_meta from the collector with module id 5, and redirect that to report.txt

```
./run_cu_query.py run "select * from in_storage.mf_storage_meta order by collection_time desc

```

The top of that file reads:

```
5,681692,2020-02-13 18:41:08,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 302)),(APP:
5,681684,2020-02-13 18:40:19,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1787)),(APP:
5,681685,2020-02-13 18:40:19,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1786)),(APP:
5,681686,2020-02-13 18:40:19,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1785)),(APP:
5,681683,2020-02-13 18:40:18,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1518)),(APP:
5,681679,2020-02-13 18:40:16,dynamic_storage,('dynamic_storage', (156, 1793)),(APP: 1793 DID:
5,681680,2020-02-13 18:40:16,dcmr_storage,dcmr_storage,(APP: 1793 DID: 156)
5,681681,2020-02-13 18:40:16,dynamic_storage,('dynamic_storage', (157, 1792)),(APP: 1792 DID:
5,681682,2020-02-13 18:40:16,dcmr_storage,dcmr_storage,(APP: 1792 DID: 157)
5,681675,2020-02-13 18:40:14,dynamic_storage,('dynamic_storage', (155, 1828)),(APP: 1828 DID:
5,681676,2020-02-13 18:40:14,dcmr_storage,dcmr_storage,(APP: 1828 DID: 155)
5,681677,2020-02-13 18:40:14,dynamic_storage,('dynamic_storage', (154, 1825)),(APP: 1825 DID:
5,681678,2020-02-13 18:40:14,dcmr_storage,dcmr_storage,(APP: 1825 DID: 154)
5,681655,2020-02-13 18:40:13,dynamic_storage,('dynamic_storage', (142, 1788)),(APP: 1788 DID:
```

```
```

The first column here is the module id and the other columns are the columns of in_storage.mf_storage_meta (seq, collection_time, type, task key, and policy_info). Note that in line 1

- the task key is ('PerfDynamicAppStorage', (1, 302))
- the policy_info is (APP: 302 DID: 1)
- I was confused about this myself and once thought the task key was: ('PerfDynamicAppStorage', (1, 302)),(APP: 302 DID: 1)
- which led me to write a query against in_storage.mf_storage_meta that returned no results (EM-36973).

If we wanted to see the last 2 storage objects from all collectors we could run this:

```
./run_cu_query.py run "select * from in_storage.mf_storage_meta order by collection_time desc

5,686225,2020-02-13 21:19:08,dynamic_storage,('dynamic_storage', (156, 1793)),(APP: 1793 DID:
5,686226,2020-02-13 21:19:08,dcmr_storage,dcmr_storage,(APP: 1793 DID: 156)
6,4213,2020-02-13 21:16:07,system_message_storage,('system_message_storage', (1,)),system_mess
6,4212,2020-02-13 21:11:03,system_message_storage,('system_message_storage', (1,)),system_mess
7,59,2020-02-13 00:52:05,system_message_storage,('system_message_storage', (7,)),system_messag
7,60,2020-02-13 00:52:05,system_message_storage,('system_message_storage', (7,)),system_messag
```

and we see mids 5, 6, and 7 all represented.

If we were interested only in BulkInterfaceStorage objects, we could run like this:

```
./run_cu_query.py run "select * from in_storage.mf_storage_meta where type='BulkInterfaceStor

5,681827,2020-02-13 18:45:12,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,681648,2020-02-13 18:40:06,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,681187,2020-02-13 18:25:06,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,674381,2020-02-13 14:25:04,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,640915,2020-02-12 18:40:06,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,640737,2020-02-12 18:35:05,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,594300,2020-02-11 15:10:04,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,594122,2020-02-11 15:05:05,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
5,593926,2020-02-11 15:00:11,BulkInterfaceStorage,('BulkInterfaceStorage', (1,)),BulkInterfac
```

inspecting storage objects with run_cu_query.py

Retrieving information about storage objects is easier than running them, which has to be done from the s-em7-core account. The easiest current way is with the get_obj2 command of run_cu_query.py. Invoking it with -h prints the following usage guide, which gives two recipes for invoking it:

```
./run_cu_query.py get_obj2 -h

usage: run_cu_query.py [-h] [--obj-attributes OBJ_ATTRIBUTES]
                       [--seq-file SEQ_FILE] [--c-time C_TIME]
```

```
                       freq mid seq_id

positional arguments:
  freq                  lf|mf|hf
  mid                   module id of collector
  seq_id                object sequence id

optional arguments:
  -h, --help            show this help message and exit
  --obj-attributes OBJ_ATTRIBUTES
                        additional attributes to retrieve
  --seq-file SEQ_FILE   a text file of sequence numbers to retrieve objects
                        for
  --c-time C_TIME       the attribute to use for collection time

        Retrieve storage object and print collection time, storage time, delay, and other att

        let's assume that we want to know about a storage object
                from the collector with mid 5
                with sequence id 1227340
                and its a PerfDynamicAppStorage object, so
                        it has attributes that we are interested in
                                data_by_obj_idx
                                derived_rates

                        and its collection time is stored in
                                collection_timestamp

        we could then run:
                ./run_cu_query.py get_obj2 mf 5 1227340 --c-time=collection_timestamp --obj-a

        and it would produce output like this:
seq_id=1227340, did=1, t_collect=1597176540, t_storage=1597176548.0, t_delay=8.0
data_by_obj_by_idx={3109: {'.0': ('0', 0, 11.744022369384766, '.0')}, 3110: {'.0': ('0', 0, 9
derived_rates=defaultdict(<type 'dict'>, {'.0': {3110: 0.0, 3111: 0.0, 3112: 0.0, 3113: 0.0,

        here is how we could retrieve the most recent 5 storage objects of type PerfDynamicAp
                ./run_cu_query.py run "select * from in_storage.mf_storage_meta where task_ke
                ./run_cu_query.py get_obj2 mf 5 0 --seq-file=f_seq_ids.txt --c-time=collectio

```

enabling the storage object inspector

If you use either of these two methods for inspecting objects:

- /opt/em7/backend/storage_obj_inspector.py - the original storage object inspector
- run_cu_query.py get_obj - a newer script that uses the same technique

you might get this message:

```
Process 255 is not allowed to run on this module type 2

```

That's because those two programs both subclass process.process_pda and run as em7 pid 255 in order to be able to execute the storage object. You will need to enable pid 255 before

these programs can work with:

```
silo_mysql -e "insert into master.map_system_settings_procs values (255,2)"

```

map_system_settings_procs has two columns: app_id, fun_id. So (255,2) is saying that process id 255 is allowed to run on DB appliances (which have a fun_id of 2).

running the storage object inspector

Now that we have some object ids to work with we are ready to start running storage_obj_inspector.py. To inspect the contents of the first interface storage object, 681827:

```
[em7admin@cdb ~]$ sudo -u s-em7-core /opt/em7/backend/storage_obj_inspector.py -o681827 -fmf
[sudo] password for em7admin:

logger instantiated
"2020-02-13 21:22:48,335","DEBUG","18582","255","inspector","73","Ok, stdout"
"2020-02-13 21:22:48,366","DEBUG","18582","255","inspector","99","setup completed"
"2020-02-13 21:22:48,511","DEBUG","18582","255","inspector","121","Data Object Retrieved"
Collector 5
Device ID 1
Colletion timestamp 2020-02-13 18:45:12


Type = <class 'storage.interface.BulkInterfaceStorage'>
Data = [(1, 1, 1581619512.0, 1581619500, 0, 1, 1, 0, '00:50:56:85:83:ee', 2, {'ifInDiscards_c
"2020-02-13 21:22:48,512","DEBUG","18582","255","proc_pda","528","tracking update: 1 255 -o68
"2020-02-13 21:22:48,512","DEBUG","18582","255","proc_pda","545","Stopped tracking process: 1
"2020-02-13 21:22:48,513","DEBUG","18582","255","proc_pda","307","Logging stopped for Storage

```

And we see the full contents of the interface object. Note the use of

```
sudo -u s-em7-core

```

It is necessary to run this command from the em7 core account or the proper permissions will be lacking.

To both pull and execute this object, we add the --store argument:

```
sudo -u s-em7-core /opt/em7/backend/storage_obj_inspector.py -o681827 -fmf -c5 --store

logger instantiated
"2020-02-13 21:24:27,302","DEBUG","20122","255","inspector","73","Ok, stdout"
"2020-02-13 21:24:27,326","DEBUG","20122","255","inspector","99","setup completed"
"2020-02-13 21:24:27,466","DEBUG","20122","255","inspector","121","Data Object Retrieved"
Collector 5
Device ID 1
Colletion timestamp 2020-02-13 18:45:12


```

```
Type = <class 'storage.interface.BulkInterfaceStorage'>
Data = [(1, 1, 1581619512.0, 1581619500, 0, 1, 1, 0, '00:50:56:85:83:ee', 2, {'ifInDiscards_c
"2020-02-13 21:24:27,467","DEBUG","20122","255","inspector","133","Storing object:
"
"2020-02-13 21:24:27,467","DEBUG","20122","255","interface","775","interface_storage storage
"2020-02-13 21:24:27,467","DEBUG","20122","255","interface","257","interface_storage: Found i
"2020-02-13 21:24:27,469","DEBUG","20122","255","interface","360","metrics enabled for if_id
"2020-02-13 21:24:27,472","DEBUG","20122","255","proc_pda","528","tracking update: 1 255 -o68
"2020-02-13 21:24:27,474","DEBUG","20122","255","proc_pda","545","Stopped tracking process: 1
"2020-02-13 21:24:27,474","DEBUG","20122","255","proc_pda","307","Logging stopped for Storage
```

Note that these lines are from the logger.debug() calls inside the storage object code:

```
"2020-02-13 21:24:27,467","DEBUG","20122","255","inspector","133","Storing object:
"
"2020-02-13 21:24:27,467","DEBUG","20122","255","interface","775","interface_storage storage '
"2020-02-13 21:24:27,467","DEBUG","20122","255","interface","257","interface_storage: Found if
"2020-02-13 21:24:27,469","DEBUG","20122","255","interface","360","metrics enabled for if_id 1
```

So, running this way allows you to see the full debug output of a storage object without having to turn on storage logging and looking in /data/logs/silo.log.

using run_cu_query.py to retrieve and run storage objects: retrieving additional attributes

The same drop file used to gather storage object ids can also be used to retrieve and run storage objects. However, you have to:

- place the file at /opt/em7/backend
- run it from the s-em7-core account

The advantage of running this utility is that you can specify additional attributes that you want printed in the report. PerfDynamicAppStorage objects, for example, store most of their data in these attributes:

- data_by_obj_idx
- derived_rates

and the normal storage_obj_inspector.py utility only prints the data attribute, which is actually set to None for in this case.

When invoked with the -h argument, it prints the following usage guide:

```
sudo -u s-em7-core /opt/em7/backend/run_cu_query.py get_obj -h

usage: run_cu_query.py [-h] [--obj-attributes OBJ_ATTRIBUTES] [--store]
                       freq mid seq_id

positional arguments:
  freq                  lf|mf|hf
  mid                   module id of collector
  seq_id                object sequence id

optional arguments:
```

```
  -h, --help            show this help message and exit
  --obj-attributes OBJ_ATTRIBUTES
                        additional attributes
  --store               runs storage object when set

        Retrieve storage object, print contents, and execute
        For this to work,
            the script has to be placed on a CDB at /opt/em7/backend and
            run via s-em7-core account

        The example command below would
                retrieve the storage object with sequence number 1284826
                from collector with mid 5
                print the additional attributes (which only exist for PerfDynamicAppStorage o
                        data_by_obj_idx
                        derived_rates
                and run the storage object while
                capturing the debug log messages to the file run1.txt

        sudo -u s-em7-core /opt/em7/backend/run_cu_query.py get_obj mf 5 1284826 --obj-attrib

```

Running that very command on my test system produces output like this:

```
did=1, t=2020-02-26 20:20:09, data=None
data_by_obj_by_idx={3076: {'.0': ('0', 0, 10.64610481262207, '.0')}, 3077: {'.0': ('0', 0, 7.
derived_rates=defaultdict(<type 'dict'>, {'.0': {3077: 0.0, 3078: 0.0, 3079: 0.0, 3080: 0.0,
storing object
"2020-02-26 20:56:26,172","DEBUG","9831","255","dynamic_app_storage","294","PerfDynamicAppSto
"2020-02-26 20:56:26,173","DEBUG","9831","255","database","173","QL: query:      SELECT
                  name, app_type,
                  IF(afo.freq IS NOT NULL, afo.freq, app.poll) AS poll,
                  comp_dev, disabled_rollup, cu_affinity,
                  align.triggered_id AS triggered_id
                FROM master.dynamic_app AS app
                LEFT JOIN (SELECT DISTINCT triggered_id FROM master.dynamic_app_comp_autoalig
                  ON app.aid = align.triggered_id
                LEFT JOIN master.dynamic_app_freq_overrides AS afo
                  ON app.aid=afo.app_id AND afo.did = 1
                WHERE aid = 299"
"2020-02-26 20:56:26,174","DEBUG","9831","255","database","174","QL: dbc id:     140074689705
"2020-02-26 20:56:26,174","DEBUG","9831","255","database","175","QL: query time: 0.0015358924
"2020-02-26 20:56:26,174","DEBUG","9831","255","database","176","QL: lastrowid:  0"
"2020-02-26 20:56:26,174","DEBUG","9831","255","database","177","QL: rowcount:   1"
"2020-02-26 20:56:26,174","DEBUG","9831","255","database","173","QL: query:      SELECT root_
"2020-02-26 20:56:26,175","DEBUG","9831","255","database","174","QL: dbc id:     140074689705
"2020-02-26 20:56:26,175","DEBUG","9831","255","database","175","QL: query time: 0.0003020763
"2020-02-26 20:56:26,175","DEBUG","9831","255","database","176","QL: lastrowid:  0"
"2020-02-26 20:56:26,175","DEBUG","9831","255","database","177","QL: rowcount:   0"
"2020-02-26 20:56:26,176","DEBUG","9831","255","database","173","QL: query:      SELECT obj_i
        FROM master.dynamic_app_objects
        WHERE app_id=299"
"2020-02-26 20:56:26,176","DEBUG","9831","255","database","174","QL: dbc id:     140074689705
"2020-02-26 20:56:26,176","DEBUG","9831","255","database","175","QL: query time: 0.0006010532
"2020-02-26 20:56:26,176","DEBUG","9831","255","database","176","QL: lastrowid:  0"
"2020-02-26 20:56:26,176","DEBUG","9831","255","database","177","QL: rowcount:   15"
"2020-02-26 20:56:26,176","DEBUG","9831","255","database","173","QL: query:      SELECT z_id,
        FROM master.dynamic_app_collection
```

```
        WHERE app_id=299 AND did=1"
"2020-02-26 20:56:26,177","DEBUG","9831","255","database","174","QL: dbc id:     140074689705
"2020-02-26 20:56:26,177","DEBUG","9831","255","database","175","QL: query time: 0.0004551410
"2020-02-26 20:56:26,177","DEBUG","9831","255","database","176","QL: lastrowid:  0"
"2020-02-26 20:56:26,177","DEBUG","9831","255","database","177","QL: rowcount:   15"
"2020-02-26 20:56:26,177","DEBUG","9831","255","common","94","App_detail: HF Data Pull: Stora
"2020-02-26 20:56:26,177","DEBUG","9831","255","perfconf_common","33"," ---        Fetching a
"2020-02-26 20:56:26,177","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,177","DEBUG","9831","255","perfconf_common","33"," ---       Getting kno
"2020-02-26 20:56:26,177","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,178","DEBUG","9831","255","database","173","QL: query:      SELECT group
        FROM master.dynamic_app_index_map
        WHERE did=1 AND app_id=299"
"2020-02-26 20:56:26,178","DEBUG","9831","255","database","174","QL: dbc id:     140074689705
"2020-02-26 20:56:26,178","DEBUG","9831","255","database","175","QL: query time: 0.0004029273
"2020-02-26 20:56:26,178","DEBUG","9831","255","database","176","QL: lastrowid:  0"
"2020-02-26 20:56:26,178","DEBUG","9831","255","database","177","QL: rowcount:   1"
"2020-02-26 20:56:26,178","DEBUG","9831","255","perfconf_common","33"," ---            Buildi
"2020-02-26 20:56:26,178","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,179","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","performance","333","map instance to ind(.0, 1
"2020-02-26 20:56:26,180","DEBUG","9831","255","perfconf_common","33"," ---                Ma
"2020-02-26 20:56:26,180","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,180","DEBUG","9831","255","common","107","mark_found - Original List: se
"2020-02-26 20:56:26,180","DEBUG","9831","255","common","108","already found: {3076: {'found'
"2020-02-26 20:56:26,180","DEBUG","9831","255","common","116","mark_found - New List: []"
"2020-02-26 20:56:26,180","DEBUG","9831","255","common","129","No objects to mark as found"
"2020-02-26 20:56:26,181","DEBUG","9831","255","common","134","set_last_collect - List: set([
"2020-02-26 20:56:26,182","DEBUG","9831","255","perfconf_common","33"," --- Executing data IN
"2020-02-26 20:56:26,182","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,183","DEBUG","9831","255","database","173","QL: query:      INSERT IGNOR
"2020-02-26 20:56:26,183","DEBUG","9831","255","database","174","QL: dbc id:     140074689705
"2020-02-26 20:56:26,183","DEBUG","9831","255","database","175","QL: query time: 0.0004730224
"2020-02-26 20:56:26,183","DEBUG","9831","255","database","176","QL: lastrowid:  0"
"2020-02-26 20:56:26,183","DEBUG","9831","255","database","177","QL: rowcount:   0"
"2020-02-26 20:56:26,184","DEBUG","9831","255","perfconf_common","33"," --- Executing descrip
"2020-02-26 20:56:26,184","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,184","DEBUG","9831","255","perfconf_common","33"," ---            Stream
"2020-02-26 20:56:26,184","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,184","DEBUG","9831","255","performance","420","Rollups disabled. Skippin
"2020-02-26 20:56:26,184","DEBUG","9831","255","perfconf_common","33"," ---                 F
"2020-02-26 20:56:26,184","DEBUG","9831","255","perfconf_common","37","PerformanceDAStorage c
"2020-02-26 20:56:26,184","DEBUG","9831","255","dynamic_app_storage","75","BaseDynamicStorage
"2020-02-26 20:56:26,184","DEBUG","9831","255","dynamic_app_storage","79","BaseDynamicStorage
"2020-02-26 20:56:26,186","DEBUG","9831","255","proc_pda","528","tracking update: 1 255 get_o
"2020-02-26 20:56:26,187","DEBUG","9831","255","proc_pda","545","Stopped tracking process: 1
```

```
"2020-02-26 20:56:26,187","DEBUG","9831","255","proc_pda","307","Logging stopped for Storage

```

Recipe: Pull and run the most recent PerfDynamicAppStorage

This recipe can be used to troubleshoot problems that occur at storage time for performance dynamic apps.

Here, we are interested in app_id 299, did 1, and did 1 is aligned with collector id 5.

```
# place run_cu_query.py in /home/em7admin on a CDB
# then login as em7admin
mkdir /tmp/run_cu
chmod a+rx /tmp/run_cu
cd /tmp/run_cu/
cp ~/run_cu_query.py .
./run_cu_query.py run "select * from in_storage.mf_storage_meta order by collection_time desc

# in this case, the module id of the CU is 5, and the most recent PerfDynamicAppStorage objec
#    5,3976835,2020-04-29 18:43:11,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 299)),
# the object sequence ID here is 3976835, so we then run
sudo -u s-em7-core ./run_cu_query.py get_obj mf 5 3976835 --obj-attributes=collection_timesta

```

Here are some of the highlights in the output of run1.txt, which we could use to debug a case where data was not making it into the raw performance tables:

```
did=1, t=2020-04-29 18:43:11, data=None
collection_timestamp=1588185780
data_by_obj_by_idx={3076: {'.0': ('0', 0, 158.4620475769043, '.0')}, 3077: {'.0': ('0', 0, 22
derived_rates=defaultdict(<type 'dict'>, {'.0': {3077: 0.0, 3078: 0.0, 3079: 0.0, 3080: 0.0,
storing object
"2020-04-29 18:48:38,087","DEBUG","9814","255","dynamic_app_storage","294","PerfDynamicAppSto
...
"2020-04-29 18:48:38,097","DEBUG","9814","255","database","173","QL: query:      INSERT IGNOR
...

```

In particular, we can decode the collection time from the second line above:

```
collection_timestamp=1588185780
```

using python like this:

```
python
Python 2.7.17 (default, Apr 15 2020, 17:20:14)
[GCC 7.5.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> from time import localtime as lt
>>> lt(1588185780)
time.struct_time(tm_year=2020, tm_mon=4, tm_mday=29, tm_hour=14, tm_min=43, tm_sec=0, tm_wday
>>> from time import gmtime as gt
>>> gt(1588185780)
time.struct_time(tm_year=2020, tm_mon=4, tm_mday=29, tm_hour=18, tm_min=43, tm_sec=0, tm_wday
>>>
```

```

```

And we see that the object was created by the collection process that started for poll time 2020-04-29 18:43:00 UTC, and it was written to in_storage.mf_storage_meta 11 seconds later both from the dump of

```
5,3976835,2020-04-29 18:43:11,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 299)),(APP:

```

and from the first line of output when running the storage object:

```
did=1, t=2020-04-29 18:43:11, data=None

```

Recipe: gather a list of N storage objects and their attributes

This is described in the syntax guide for run_cu_query.py's get_obj2 command, in this section above: [inspecting storage objects with run_cu_query.py]

Below, we walk through that recipe, running it on a test system and quoting the full output. In that syntax guide, these commands were offered:

```
./run_cu_query.py run "select * from in_storage.mf_storage_meta where task_key like '%PerfDyna
./run_cu_query.py get_obj2 mf 5 0 --seq-file=f_seq_ids.txt --c-time=collection_timestamp --obj

```

If we save the full output of the first query before extracting just the sequence ids, we would have:

```
./run_cu_query.py run "select * from in_storage.mf_storage_meta where task_key like '%PerfDyna
cut -d, -f2 < report_meta.txt > f_seq_ids.txt
./run_cu_query.py get_obj2 mf 5 0 --seq-file=f_seq_ids.txt --c-time=collection_timestamp --obj

```

report_meta.txt looks like this:

```
cat report_meta.txt
5,1239332,2020-08-12 15:20:20,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1785)),(APP:
5,1239331,2020-08-12 15:20:20,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1787)),(APP:
5,1239330,2020-08-12 15:20:20,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1786)),(APP:
5,1239329,2020-08-12 15:20:19,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 1515)),(APP:
5,1239328,2020-08-12 15:20:17,PerfDynamicAppStorage,('PerfDynamicAppStorage', (1, 993)),(APP:

```

Each type of storage object can have different attributes where data is arranged. In the code for PerfDynamicAppStorage, we see:

```
class PerfDynamicAppStorage(BaseDynamicStorage, storage.RollupsSwitchMixin):
    #TODO: further simplify the init signature
    #      the shape of data_by_obj_idx is still defined by config app holdovers
    def __init__(
        self,
        did,
```

```
        app_id,
        collection_timestamp,
        data_by_obj_by_idx,
        derived_rates,
        meta,
        found_collection_objects,
        time_details  #deprecated - no longer needed
    ):
        BaseDynamicStorage.__init__(self, None)
        self.did = did
        self.root_did = None
        self.app_id = app_id
        self.collection_timestamp = collection_timestamp
        self.data_by_obj_by_idx = data_by_obj_by_idx
        self.derived_rates = derived_rates
...

```

In this case

- collection_timestamp - is the time when data was collected
- data_by_obj_by_idx - is a multi layered dictionary of gathered data
- derives_rates - I believe this is the result of delta calculations on any counter values collected (current values minus the values from the last collection)

In the third command above, we use the arguments

- --ctime=collection_timestamp to tell run_cu_query.py which attribute stores the collection time of the data
- --obj-attributes=data_by_obj_by_idx,derived_rates to tell run_cu_query.py to decode this data and print it

The output file run1.txt contains, for every storage object retrieved

- a summary line that measures the delay from data collection to storage object creation (the storage object creation time is taken from in_storage.X_storage.collection_time). Extracting only those from run1.txt we have (showing 10-11 second delays):

```
grep did= run1.txt

seq_id=1229550, did=1, t_collect=1597189260, t_storage=1597189271.0, t_delay=11.0
seq_id=1229549, did=1, t_collect=1597189260, t_storage=1597189271.0, t_delay=11.0
seq_id=1229548, did=1, t_collect=1597189260, t_storage=1597189270.0, t_delay=10.0
seq_id=1229547, did=1, t_collect=1597189260, t_storage=1597189270.0, t_delay=10.0
seq_id=1229545, did=1, t_collect=1597189260, t_storage=1597189270.0, t_delay=10.0

```

- a line in the format "attribute=value" for every attribute provided via --obj-attributes

On our test system, the full run1.txt was:

```
seq_id=1229550, did=1, t_collect=1597189260, t_storage=1597189271.0, t_delay=11.0
data_by_obj_by_idx={3139: {'.0': ('121160', 0, 11.711835861206055, '.0')}, 3140: {'.0': ('908
derived_rates=defaultdict(<type 'dict'>, {'.0': {3156: 3.0, 3153: 0.016715558998839697, 3154:
seq_id=1229549, did=1, t_collect=1597189260, t_storage=1597189271.0, t_delay=11.0
data_by_obj_by_idx={3136: {'.0': ('6356326', 0, 16.85500144958496, '.0')}, 3138: {'.0': ('635
derived_rates=defaultdict(<type 'dict'>, {'.0': {3136: 0.0, 3138: 0.0, 3125: 0.0, 3126: 0.0,
seq_id=1229548, did=1, t_collect=1597189260, t_storage=1597189270.0, t_delay=10.0
data_by_obj_by_idx={3109: {'.0': ('0', 0, 34.15799140930176, '.0')}, 3110: {'.0': ('0', 0, 23
derived_rates=defaultdict(<type 'dict'>, {'.0': {3110: 0.0, 3111: 0.0, 3112: 0.0, 3113: 0.0,
seq_id=1229547, did=1, t_collect=1597189260, t_storage=1597189270.0, t_delay=10.0
data_by_obj_by_idx={3168: {'.1': ('1', 0, 68.91703605651855, '.1'), '.2': ('1', 0, 68.9170360
derived_rates=defaultdict(<type 'dict'>, {'.0': {3174: 18.0, 3171: 0.009032970000589557, 3157
seq_id=1229545, did=1, t_collect=1597189260, t_storage=1597189270.0, t_delay=10.0
data_by_obj_by_idx={3076: {'.0': ('0', 0, 47.66201972961426, '.0')}, 3077: {'.0': ('0', 0, 12
derived_rates=defaultdict(<type 'dict'>, {'.0': {3077: 0.0, 3078: 0.0, 3079: 0.0, 3080: 0.0,
```

Recipe: gather the N most recent storage objects with the given task key pattern

This script can be used to automate that process:

Error loading the extension!

Recipe: run one of the storage objects you just gathered

Continuing the above recipe, once you have a set of storage objects identified by sequence number and you know the CU's mid, you can pull and execute the storage object with (again, running from the CDB)

```
sudo -u s-em7-core ./run_cu_query.py get_obj mf <cu_mid> <sequence number>  --store > run_so.t

```

Here is an example of what the output looks like in run_so.txt:

```
did=129822, t=2021-01-05 15:51:06, data=(129822, 64261, 1016, 1609861860, [u'SqlInstance.PROD
storing object
"2021-01-05 16:00:44,212","DEBUG","93337","255","component","1235","dcm_avail_storage 'store'
"2021-01-05 16:00:44,212","DEBUG","93337","255","component","131","did=129822"
"2021-01-05 16:00:44,214","DEBUG","93337","255","database","173","QL: query:      SELECT id a
        FROM master_dev.legend_device
        WHERE id = 129822
        UNION
        SELECT id as did, IF((active & 2)=0, 1, 0) AS avail, 1 AS vanished
        FROM master_dev.legend_device_vanished
        WHERE id = 129822"
"2021-01-05 16:00:44,214","DEBUG","93337","255","database","174","QL: dbc id:     14033587014
"2021-01-05 16:00:44,214","DEBUG","93337","255","database","175","QL: query time: 0.001301050
"2021-01-05 16:00:44,214","DEBUG","93337","255","database","176","QL: lastrowid:  0"
"2021-01-05 16:00:44,214","DEBUG","93337","255","database","177","QL: rowcount:   1"
"2021-01-05 16:00:44,217","DEBUG","93337","255","database","173","QL: query:      SELECT cdm.
                FROM master_dev.component_dev_map cdm
                JOIN master_dev.legend_device ld
                ON ld.id = cdm.parent_did
                WHERE cdm.component_did= 129822
                UNION
```

```
                SELECT cdm.parent_did AS did, 1 as vanished, IF((ld.active & 2)=0, 1, 0) AS a
                FROM master_dev.component_dev_map cdm
                JOIN master_dev.legend_device_vanished ld
                ON ld.id = cdm.parent_did
                WHERE cdm.component_did= 129822"
"2021-01-05 16:00:44,217","DEBUG","93337","255","database","174","QL: dbc id:     14033587014
"2021-01-05 16:00:44,218","DEBUG","93337","255","database","175","QL: query time: 0.002155065
"2021-01-05 16:00:44,218","DEBUG","93337","255","database","176","QL: lastrowid:  0"
"2021-01-05 16:00:44,218","DEBUG","93337","255","database","177","QL: rowcount:   1"
"2021-01-05 16:00:44,218","DEBUG","93337","255","component","163","all ancestors of 129822 av
"2021-01-05 16:00:44,220","DEBUG","93337","255","database","173","QL: query:      SELECT cdm.
        FROM master_dev.component_dev_map cdm
        JOIN master_dev.legend_device ld
          ON ld.id = cdm.component_did
        WHERE cdm.discovered_by_aid=1016
          AND cdm.parent_did=129822
          AND (ld.active & 9) = 0
          AND ld.pid != 0
        UNION
        SELECT cdm.component_did AS did, cdm.unique_id, ld.active, IF(cdm.merge_did!=0, 1, 0)
        FROM master_dev.component_dev_map cdm
        JOIN master_dev.legend_device_vanished ld
          ON ld.id = cdm.component_did
        WHERE cdm.discovered_by_aid=1016
          AND cdm.parent_did=129822
          AND (ld.active & 9) = 0
          AND ld.pid != 0"
"2021-01-05 16:00:44,220","DEBUG","93337","255","database","174","QL: dbc id:     14033587014
"2021-01-05 16:00:44,221","DEBUG","93337","255","database","175","QL: query time: 0.001534938
"2021-01-05 16:00:44,221","DEBUG","93337","255","database","176","QL: lastrowid:  0"
"2021-01-05 16:00:44,221","DEBUG","93337","255","database","177","QL: rowcount:   3"
"2021-01-05 16:00:44,222","DEBUG","93337","255","component","1289","Found 0 unavailable child
"2021-01-05 16:00:44,227","DEBUG","93337","255","proc_pda","528","tracking update: 1 255 get_
"2021-01-05 16:00:44,228","DEBUG","93337","255","proc_pda","545","Stopped tracking process: 1
"2021-01-05 16:00:44,228","DEBUG","93337","255","proc_pda","307","Logging stopped for Storage

```

Notice how you can see the literal text of each query that was run.

Postmortem Examples

- NRB_vm_devices_not_vanishing: in this case, run_cu_query.py was used to retrieve a DCM storage object and debug why a device that should have been vanishing was not.
- Sasktel Merge Device Not Collecting: in this case the storage object inspector was used to zero in on the main problem, that the storage objects were being created but with no data:
- 2020-08-11 Next Steps: cast a wider net and retrieve storage object delays for app 2153 for 48 hours: here "run_cu_query.py get_obj2": command was used to search for long collection-to-storage-object creation delays

worker processes with high CPU utilisation

This is a red flag. If you run top, and you see data pull worker processes moving up the list, then the storage object it is executing has gone off the deep end. You would normally expect a worker process to be IO bound and not CPU bound, as it should be running storage queries and waiting on the results.

look for timeouts

These can devestate data pull performance. Enable storage logging and then "grep -i timeout silo.log" and look for storage object timeouts. The Row Id quoted is the seq_id you need to retrieve and execute the object with the object inspector, so you can see why the storage object is taking so long from its debug messages.

slow storage object logging

See this TSG section: Rows-Behind Troubleshooting Guide | Data Pull Slow Storage Log

Theory

in_storage.X_storage tables

Collection processes create storage objects and append them to the tables:

- in_storage.lf_storage
- in_storage.mf_storage
- in_storage.hf_storage

which are all defined equivalently to the mf case as:

```
CREATE TABLE `mf_storage` (
  `seq` bigint(11) unsigned NOT NULL AUTO_INCREMENT,
  `did` int(11) unsigned NOT NULL,
  `collection_time` datetime NOT NULL,
  `data` mediumblob NOT NULL,
  PRIMARY KEY (`seq`,`collection_time`),
  KEY `did` (`did`)
) ENGINE=InnoDB  DEFAULT CHARSET=latin1
PARTITION BY LIST (TO_DAYS( collection_time ))
(PARTITION P0 VALUES IN (0) ENGINE = InnoDB);

```

the data column contains a python pickle of the storage object. Another set of tables:

- in_storage.lf_storage_meta
- in_storage.mf_storage_meta
- in_storage.hf_storage_meta

are all defined equivalently to the mf case as:

```
CREATE TABLE `mf_storage_meta` (
  `seq` bigint(11) unsigned NOT NULL,
  `collection_time` datetime NOT NULL,
  `type` text NOT NULL,
  `task_key` text NOT NULL,
  `policy_info` text NOT NULL,
  PRIMARY KEY (`seq`, `collection_time`)
) ENGINE=InnoDB  DEFAULT CHARSET=latin1
PARTITION BY LIST (TO_DAYS(collection_time))
(PARTITION P0 VALUES IN (0) ENGINE = InnoDB);
```

The utility run_cu_query.py can be useful in running queries against a large set of CUs. For example, this command line, when run on the CDB:

```
./run_cu_query.py run "select seq, collection_time, type, task_key, policy_info from in_storag
sort -t, -k3 dump_hf_meta.txt > dump_hf_meta_sorted.txt
```

would gather, from all CUs in the system, the last 10000 rows of in_storage.hf_storage_meta. Here is some sample output from this run on a test system:

```
head -10 dump_hf_meta_sorted.txt
5,7438,2023-10-16 00:00:01,appliance_uuid_hash_storage,appliance_uuid_hash_storage,appliance_
3,101462,2023-10-16 00:00:02,appliance_uuid_hash_storage,appliance_uuid_hash_storage,applianc
4,7652,2023-10-16 00:00:02,appliance_uuid_hash_storage,appliance_uuid_hash_storage,appliance_
5,7439,2023-10-16 00:00:06,needs_restart_storage,needs_restart_storage,needs_restart_storage
3,101463,2023-10-16 00:00:29,needs_restart_storage,needs_restart_storage,needs_restart_storag
5,7440,2023-10-16 00:00:44,system_status_output_storage,system_status_output_storage,system_s
3,101465,2023-10-16 00:01:01,event_dynamic,1,event_dynamic
3,101464,2023-10-16 00:01:01,system_status_output_storage,system_status_output_storage,system
4,7653,2023-10-16 00:01:09,needs_restart_storage,needs_restart_storage,needs_restart_storage
3,101466,2023-10-16 00:01:21,event_dynamic,2,event_dynamic

```

The first column is the module id of the CU, and the remaining columns are from the query against in_storage.hf_storage_meta.

Data Pull will spawn a worker process to consume and store queued storage objects called data_pull_pool_proc.py. You can see this using top or ps.

```
196531 s-em7-c+  20   0 1721008   1.3g   9544 R 100.0  0.5   0:09.30 /usr/bin/python2.7 /usr/l
```

Metrics

In some cases there is complaining from the snmp_agent about no data for metrics group (lf_pull, mf_pull, hf_pull). Example:

```
"2021-01-04 23:02:17,074","WARNING","25148","140265919166208","snmp_agent","181","  Failure: s
```

```
"2021-01-04 23:02:17,074","WARNING","25148","140265919166208","snmp_agent","181","  Unhandled
"2021-01-04 23:02:17,074","WARNING","25148","140265919166208","snmp_agent","181","  Unhandled
"2021-01-04 23:02:17,074","WARNING","25148","140265919166208","snmp_agent","181","  Traceback
```

Running the metrics script to retrieve data pull stats also exposes the issue:

```
[root@se2-swg-db-ha-01 ~]# /opt/em7/bin/python -m silo_db.data_pull.pull_metrics lf
Unhandled error in Deferred:
Unhandled Error
Traceback (most recent call last):
Failure: silo_metrics.collector.query_client.QueryException: Query error: No data found for me
```

When a Data Pull process is started up, the corresponding metric group for the frequency is created unless the 'instrumentation_relay' option is set to 'off' for the Data Pull config. If this option has been set in /etc/silo.conf, this disables metrics in data pull. If metrics are desired, it will need to be removed from silo.conf and each of the data pull processes (em7_lf/mf/hf_pulld.service) needs to be restarted.

Post Mortem Articles

2025-03-12 EM-72817: Lots of "Storage Object Failure due to ERROR STORING DATA." after an upgrade from 12.1.2 to 12.3.1

Link: EM-72817: Lots of "Storage Object Failure due to ERROR STORING DATA." after an upgrade from 12.1.2 to 12.3.1 CLOSED

From the description:

```
Observed Behavior: A lot of "Storage Object Failure due to ERROR STORING DATA" messages in Sy

Expected Behavior: There should not be any issues with data pull processes after an upgrade.

Summary of troubleshooting steps taken so far and the current status:



Enabled Storage logging and captured debug log and storage log file that covers the time the

Those messages seem to occur very frequently, multiple times a minute.

Stack was upgraded from 12.1.2 to 12.3.1 on 3/6/2025, however the issue was noticed just now.



Here is an example message:
    2025-03-12T19:26:59.672291+00:00 localhost silo.em7_hfpulld.57[3442346]: INFO storage_tas
    Type: event_internal DID: 371996 Policy Info: event_internal Collector ID: 202 Row ID: 23

Storage log contains messages like below:
    2025-03-12T22:37:45.095958+00:00 localhost silo.em7_hfpulld_storage_process: StorageProc1
    #012Traceback (most recent call last):#012  File "/opt/em7/lib/python3.11/storage/storage
    store_default#012  File "/opt/em7/lib/python3.11/internal.py", line 116, in store#012  Fi
    line 58, in db_utf8_by_flag#012  File "/opt/em7/lib/python3.11/MySQLdb/connections.py", l
```

```
    MySQLdb.ProgrammingError: (2014, "Commands out of sync; you can't run this command now")

There are different variations of storage object errors. Here is another example:
2025-03-10T03:55:03.595285+00:00 localhost silo.em7_hfpulld_storage_process: StorageProc1[345
(1654, 1741578840, b'HTTP/1.1 401 Unauthorized\r\nServer: nginx\r\nDate: Mon, 10 Mar 2025 03:
Basic realm="EM7 API Server"\r\nX-Powered-By: ScienceLogic, Inc. - EM7 API/Integration Server
Valid authentication credentials not provided\r\nX-EM7-status-code: NO_AUTH\r\nVary: Accept-E
text/xml;charset=UTF-8\r\nTransfer-Encoding: chunked\r\nConnection: keep-alive\r\nX-EM7-Imple
private, no-cache, must-revalidate, no-store, max-age=0, post-check=0, pre-check=0\r\nVary: A
elemtype="href"/><link URI="/api/account" description="Get/Update/Add/Delete User Accounts" e
description="Add Alerts" elemtype="href"/><link URI="/api/appliance" description="Get/Update
URI="/api/cbqos_metric" description="Get/Update CBQoS Metrics" elemtype="href"/><link URI="/a
<link URI="/api/collection_label" description="Get Collection Label" elemtype="href"/><link U
description="Get/Update/Add/Delete Collector Groups" elemtype="href"/><link URI="/api/contact
elemtype="href"/><link URI="/api/custom_attribute" description="Get/Update/Add/Delete Custom
URI="/api/data_performance" description="Get collected rollup performance data across devices
URI="/api/device" description="Get/Update/Add/Delete Devices and Get Collected Data" elemtype
description="Get Device Classes" elemtype="href"/><link URI="/api/device_group" description="
elemtype="href"/><link URI="/api/discovery_session" description="Get/Update/Add/Delete Device
or apply discover_session resource to initiate discovery" elemtype="href"/><link URI="/api/dy
description="ScienceLogic Library alignment with Execution Environments" elemtype="href"/><li
Event Categories" elemtype="href"/><link URI="/api/execution_environment" description="Get Ex
URI="/api/interface_metric" description="Get/Update Interface Metrics" elemtype="href"/><link
description="Get/Update/Add/Delete Organizations" elemtype="href"/><link URI="/api/powerpack"
elemtype="href"/><link URI="/api/relationship" description="Get Relationships" elemtype="href
description="Get/Update/Add/Delete Schedules" elemtype="href"/><link URI="/api/silo_library"
or apply filestore/system_patch resource to register" elemtype="href"/><link URI="/api/system
elemtype="href"/><link URI="/api/system_patch_stage" description="Get staged system patch, or
elemtype="href"/><link URI="/api/system_threshold" description="Get/Update System Thresholds"
description="Get/Update/Add/Delete Theme Resources" elemtype="href"/><link URI="/api/threshol
description="Get/Update/Add/Delete Tickets" elemtype="href"/><link URI="/api/ticket_category"
description="Get Ticket Chargeback entries" elemtype="href"/><link URI="/api/ticket_log" desc
description="Get Ticket Notes/Add Attachments to Ticket Notes" elemtype="href"/><link URI="/a
description="Get/Update/Add/Delete Custom Ticket States" elemtype="href"/><link URI="/api/ven
 Commands out of sync; you can't run this command now

Need assistance from Engineering to identify the exact impact and workaround/fix for the issu
SRE is planning to rollout 12.3.x very soon, with CDW stacks planned to be upgraded starting
See logs attached.
Thank you.
-Leo
```

This comment describes one cause of the “Commands out of sync; you can't run this command now" errors from mariadb: EM-72817: Lots of "Storage Object Failure due to ERROR STORING DATA." after an upgrade from 12.1.2 to 12.3.1 CLOSED

```
I have run into “Commands out of sync; you can't run this command now“ before and written a TS
before running the next query.
```

The article linked in that comment is here: MySQL Gotchas | 2023 01 18 Commands out of sy nc; you can't run this command now

Here is a root cause analysis of this: Skylar: Troubleshooting Data Pull w/ Skylar Enabled

2024-10-11 EM-70585: Investigate "not processing deprecated messages events"

ts_guide_to_jira_link

Link: EM-70585: Investigate "not processing deprecated messages events" CLOSED

From the description:

```
Observed Behavior:
The customer has updated all appliances to 12.2.3 and they are still getting alerts for "not p

Expected Behavior:
updating the collectors to 12.2.3 should have stopped those mesages.

Summary of troubleshooting steps taken so far and the current status:
(Support Engineers need to provide a detailed summary of what steps have been taken so far and
Validated that the appliances were updated correctly. Reviews the Associated JIRA and KB Artic
```

This issue (and how to mitigate it) is described in this article: [CRITICAL] Seeing "NOT PROCESSING DEPRECATED MESSAGE FORMAT" events after upgrading to 12.2.3 from 12.1.x (Pre-12.1.2) or 12.2.0

2024-06-11 EM-65237: aws-kd-mc-eads-elv-2-sl1-de-01-prod - dl09pavsl1009 medium frequency is 128550 behind (threshold: 120000)

ts_guide_to_jira_link

Link: KD-MC-EADS-SaaS-PROD-NA-ELV2 MF rows behind

From the description:

```
Observed Behavior: Saw one long running query which is waiting on empty queue. Need to unders
Expected Behavior:  Need assistance in identifying the issue.

Summary of troubleshooting steps taken so far and the current status:
Checked the current MF count:
Every 5.0s: paste aggregate_label aggregate ; echo ; paste collectors low_behind medium_behin

medium 1177166
low 68352
high 2

dl09pavsl1003 6045 119452 0
dc04pavsl1007 5981 101268 0
dc04pavsl1008 5944 104352 0
dc04pavsl1006 5920 101979 0
dl09pavsl1009 5918 117708 0
dl09pavsl1012 5859 98362 2
dc04pavsl1009 5840 97032 0
dl09pavsl1001 5830 104617 0
dl09pavsl1004 5809 114692 0
dl09pavsl1002 5798 106518 0
dl09pavsl1011 3847 102936 0
dc04pavsl1014 1593 2558 0
dc04pavsl1016 1523 4 0
dc04pavsl1017 646 1 0
```

```
dc04pavsl1011 269 1540 0
dl09pavsl1008 178 11 0
dl09pavsl1007 173 151 0
dl09pavsl1006 162 11 0
dl09pavsl1010 161 12 0
dc04pavsl1012 143 11 0
dc04pavsl1010 142 415 0
dc04pavsl1013 139 2000 0
dc04pavsl1015 121 1238 0
dl09pavsl1005 115 6 0
ip-10-152-2-178.ec2.internal 94 4 0
dc04davsl1004.wellpoint.com 51 146 0

Checked for long running queries:


Every 1.0s: silo_mysql -BNe "show processlist" | grep -v Sleep | sort -k6 -rn | head

315667314       event_scheduler localhost       NULL    Daemon  1003292 Waiting on empty queu
337225939       clientdbuser    10.152.2.100:57488      mysql   Query   2       update  INSER
337225938       clientdbuser    10.152.2.100:57486      mysql   Query   1       update  INSER
337225935       clientdbuser    10.152.2.100:57480      mysql   Query   1       update  INSER
337225932       clientdbuser    10.152.2.100:57474      mysql   Query   1       update  INSER
337225930       clientdbuser    10.152.2.100:57472      mysql   Query   1       update  INSER
337260602       clientdbuser    10.152.2.100:51308      NULL    Query   0       starting
337260160       clientdbuser    10.152.2.100:50848      mysql   Query   0       update  INSER
337225941       clientdbuser    10.152.2.100:57492      mysql   Query   0       update  INSER
337225940       clientdbuser    10.152.2.100:57490      mysql   Query   0       update  INSER

As we can see below query is running for a very long time:
315667314       event_scheduler localhost       NULL    Daemon  1003324 Waiting on empty queu

From the looks of it the query is initiated from the UI itself. I tried to identify the sched

But wasn’t able to do so.

Automation task:


[root@ip-10-152-2-100 ~]# _ran, avg(task_duration) as avg_time, max(task_duration) as max_tim
+-------+-----------------+--------------+----------+-----------+
| ap_id | automations_ran | avg_time     | max_time | sum_total |
+-------+-----------------+--------------+----------+-----------+
|    93 |             346 |   3.17612803 |   5.5568 | 1098.9403 |
|    94 |             301 |   5.56478738 |   7.4005 | 1675.0010 |
|   132 |             175 |   5.55841257 |   6.8941 |  972.7222 |
|    95 |              12 |  15.24815833 |  21.4912 |  182.9779 |
|   131 |              11 |   5.95038182 |  10.8480 |   65.4542 |
|   126 |               1 | 341.11450000 | 341.1145 |  341.1145 |
|   127 |               1 | 302.51080000 | 302.5108 |  302.5108 |
|    71 |               1 |  29.66330000 |  29.6633 |   29.6633 |
|    97 |               1 |  19.66060000 |  19.6606 |   19.6606 |
+-------+-----------------+--------------+----------+-----------+
automation tasks are completed within the max time.

RDS Usage:

The cases for MF get raised daily at around 11:40am AEST.
```

```

Therefore I checked processes like Nightly update:
```

The long running MariaDB/RDS thread that uses the internal user “event_scheduler” is a normal occurance, and does not cause problems by consuming CPU usage or holding row locks that slow down other queries.

This should be investigated as a normal mf rows behind case.

In this analysis section, four long running queries were identified from dumps of the mysql process lists: KD-MC-EADS-SaaS-PROD-NA-ELV2 MF rows behind | 2024 07 02 Analysis of data gathered by watch_locks.sh: source of long running ...

After killing those, the CPU usage on the RDS instance (viewed via Amazon monitoring interface, not by watching any dynamic apps from the device that ateam modelles) started trending down as well as rows behind. Over the next few days, the rows behind caught up.

The longest running query (for almost a day) was a subscription billing query. If this stack goes into rows behind again, we recommend using mysql process list to see if that query is playing a role and if so, digging further into how. This section of the article gives the query details: KD-MC-EADS-SaaS-PROD-NA-ELV2 MF rows behind | 2024 07 02 Next Steps: why does telemetry query run for so long

2024-05-07 EM-64857: Escalation for case 00435646

ts_guide_to_jira_link

Link: EM-64857: Escalation for case 00435646 CLOSED

From the description:

```
Which troubleshooting doc(s) were reviewed: Discussed this with team-what:
Customer Timezone: Denver time
Observed Behavior: We are seeing error exceptions.ValueError: unsupported pickle protocol: 3
Expected Behavior: Resolve the error
Summary of troubleshooting steps taken so far and the current status:
Collection has stopped for multiple collectors and here is the information that I collected o
Case 00435968.docx
Case 00435968.docx
06 May 2024, 08:00 PM

We are able to ping the device from the collector and also get the snmp walk output.Running t

Credential test is also not getting completed

    Current MF.

Mon May  6 22:56:37 UTC 2024
medium  963872
low     6341
high    3
```

```
prsl-mbdcol02   6334    729073  3
prsl-hrdcol02   0       109761  0
prsl-mbdcol01   7       79351   0
prsl-mbdcol03   0       36308   0
prsl-hrdcol01   0       7414    0
ip-10-204-48-45.ec2.internal    0       1902    0
prsl-mbmcol01   0       63      0
prsl-hrmcol01   0       0       0

We also seeing below error:
2024-05-06T23:04:05.990327+00:00 localhost silo.em7_mfpulld.78[1476703]: CRITICAL main.emit 1
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/python/log.py", line
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/python/log.py", line
File"/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/python/context.py", li
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/python/context.py", l
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/internet/posixbase.py
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/internet/process.py",
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/internet/fdesc.py", l
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/internet/process.py",
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/internet/process.py",
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/internet/protocol.py"
File "/opt/em7/backend/silo_db/data_pull/store.py", line 205, in outReceived#012    #012
File "/opt/em7/lib/python/Twisted-12.1.0-py2.7-linux-x86_64.egg/twisted/protocols/basic.py",
File "/opt/em7/backend/silo_db/data_pull/store.py", line 175, in stringReceived#012    #012
File "/opt/em7/backend/silo_db/data_pull/store.py", line 488, in handle_incoming_storage_resu
File "/opt/em7/backend/silo_db/data_pull/storage_proc_protocol.py", line 78, in parse_storage

Seems like 12.2.1 is on the portal and currently it has been imported:

image-20240507-000520.png

Discussed this with team-what:
team-what suggested to install 12.1.1 drop files to fix the issue.
```

2024-04-04 EM-64173: v12.1.1 SL1 upgrade issues - Singledigits03-prod

ts_guide_to_jira_link

Link: EM-64173: v12.1.1 SL1 upgrade issues - Singledigits03-prod CLOSED

From the description:

```
Observed Behavior:
Following a system update, deployment to the collectors did not appear to be complete, at fir
Rows behind keeps going up, HF queues don't seem to be doing anything, there are many outstan
  but what was done to workaround this issue is not immediately apparent

Expected Behavior: Rows behind process
Summary of troubleshooting steps taken so far and the current status:

Been looking at the issue with Matt from SRE for a while. We found this previous recent examp
did to mitigate the issue. This is on the verge of being classified as an outage
```

2024-03-29 EM-64099: Degraded performance in HF data pull due to task_key "0" in storage objects

ts_guide_to_jira_link

Link: EM-64099: Degraded performance in HF data pull due to task_key "0" in storage objects CLOSED

From the description:

```
Observed Behavior:
HF data pull lags due to large backlog of deferred storage objects with task_key=0 (essentiall
storage objects in question appear to be an internal event that alludes to some sort of code o
deleting storage objects in question caused HF data pull to progress at an acceptable rate

Expected Behavior:
HF data pull should run optimally at all times
```

From this comment EM-64099: Degraded performance in HF data pull due to task_key "0" in storage objects CLOSED

```
If this issue comes back (task key 0 storage objects, of some event type like event_internal
ENV_OPT_QUEUE="hf" ENV_OPT_STORE="--obj-attributes=matching_ders,event_id,event_defs,record"

For more information on what all this does and how it works, see my new write up here:

The above command line will:
    when run on the CDB
    reach out to all CU’s and
    query their in_storage.hf_storage_meta table to look for storage objects with a task key
    retrieve information about the last 10 such, printing their
    event attributes

If we know the module id of a specific CU that the task key 0 objects are accumulating at, it
ENV_OPT_QUEUE="hf" ENV_OPT_STORE="--obj-attributes=matching_ders,event_id,event_defs,record"

You will need to drop these two files on the CDB before trying to run this:
```

2022-11-10 EM-54550 Bandwidth Billing Isn't Working

Links: EM-54550: Bandwidth Billing Isn't Working CLOSED

ts_guide_to_jira_link

This Jira issue shows how to use the storage object inspector to trouble shoot storage objects. This troubleshooting guide was written to document the procedure used in EM-54550: Interface Bandwidth Billing | using the storage object inspector

Known Issues

Silo Log Messages

Introduction

analysis script

Log Pattern Analysis

stanza:timeout

These examples are taken from this post mortem: [2020-02-27 storage object logging revealed timeouts]

```
"2020-02-27 12:47:07,275","DEBUG","55681","57","main","531","Storage Object Failure due to TIM
"2020-02-27 12:48:07,810","DEBUG","55681","57","store","516","Pool Process 3 restarting due to
"2020-02-27 12:48:07,819","DEBUG","55681","57","store","273","  Exit due to storage object tim
"2020-02-27 12:48:07,820","DEBUG","55681","57","main","531","Storage Object Failure due to TIM
"2020-02-27 12:49:08,368","DEBUG","55681","57","store","516","Pool Process 12 restarting due t
"2020-02-27 12:49:08,375","DEBUG","55681","57","store","273","  Exit due to storage object tim

```

Storage object timeouts can kill the performance of data pull and suppressing the events causing timeouts are a major mitigation strategy in rows behind scenarios.